DATA STRUCTURES & ALGORITHMS

End-Semester Examination — Complete Study Notes

Coverage: All 8 units in your syllabus: definitions, key concepts, algorithms, operations, applications, examples, and time complexities.

How to use: Learn definitions first, practise writing algorithms without looking, then revise the complexity tables and long-answer questions.

Prepared from the syllabus visible in your uploaded image.

Quick Revision: What to Learn First

• **High priority:** stack applications and expression conversion; recursion; queue variants; linked-list operations; binary-tree traversals; BST insertion/deletion; AVL rotations; hashing collisions; BFS and DFS.

• **For algorithm questions:** write the idea, pseudocode, one worked example, and best/average/worst time complexity where applicable.

• **For comparison questions:** give at least 4 differences in a table (e.g., array vs linked list, BFS vs DFS, AVL vs Red-Black, linear vs binary search).

• **Important notation:** O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, $O(n^2)$ quadratic.

Complexity notation

Time complexity describes how running time grows with input size n. **Space complexity** describes extra memory used. Big-O gives an asymptotic upper bound; Ω gives a lower bound; Θ gives a tight bound.

Typical growth order (fastest to slowest): O(1), O(log n), O(n), O(n log n), $O(n^2)$, O(2■), O(n!).

UNIT 1 — Introduction to Data Structures

Weightage 10%. Focus on classifications, operations, arrays, structures, pointers, dynamic allocation, and algorithm analysis.

1. Data structure: definition

A **data structure** is a way of organizing and storing data in memory so that it can be accessed, updated, and processed efficiently. Examples include arrays, linked lists, stacks, queues, trees, graphs, and hash tables.

2. Classification of data structures

• **Primitive:** built-in basic types such as int, char, float, boolean and pointer.

• **Non-primitive:** constructed from primitive types. Linear structures arrange elements sequentially (array, linked list, stack, queue); non-linear structures represent hierarchies or networks (tree, graph).

• **Static:** size is fixed before or during allocation and does not grow automatically (typical fixed array). **Dynamic:** size can change during execution (linked list, dynamically resized array).

3. Common data-structure operations

Traversal visits elements; **searching** finds a target; **insertion** adds an element; **deletion** removes an element; **sorting** orders elements; **merging** combines structures.

4. Algorithm and algorithm analysis

An **algorithm** is a finite, unambiguous sequence of steps that accepts input, processes it, and produces output. A good algorithm has input, output, definiteness, finiteness, and effectiveness.

Time complexity measures growth in operations as input grows. **Space complexity** measures memory use, including auxiliary space. Best case is the most favourable input, average case is expected behaviour, and worst case is the maximum work for inputs of size n.

5. Arrays and memory representation

An **array** stores same-type elements in contiguous memory locations, allowing direct indexed access. For a one-dimensional array A with base address BA, element size w, and lower bound LB:

Address(A[i]) = BA + (i − LB) × w.

Example: BA = 1000, w = 4 bytes, LB = 0; address of A[3] = 1000 + 3×4 = 1012.

• Advantages: O(1) indexed access, compact storage, cache-friendly.

• Limitations: fixed capacity for ordinary static arrays; insertion/deletion in the middle requires shifting, typically O(n).

6. Structures, self-referential structures, unions, pointers

A **structure** groups fields of possibly different data types under one name. A **self-referential structure** contains a pointer to another object of the same structure type; it is used to build linked lists and trees. A **union** allows different members to share the same memory region, so typically only one member's stored value is meaningful at a time.

A **pointer** stores a memory address. Dereferencing a pointer accesses the object at that address. Null pointers point to no valid object; dangling pointers refer to memory that is no longer valid.

7. Dynamic memory allocation

Dynamic allocation obtains memory at runtime. In C, malloc() allocates uninitialised bytes, calloc() allocates and zero-initialises, realloc() changes allocation size, and free() releases allocated memory. Check allocation results and release memory exactly once to avoid leaks and invalid frees.

Exam-ready answer: static vs dynamic allocation

Static allocation has size determined in advance and is simple to manage but less flexible. Dynamic allocation occurs at runtime, supports variable-sized structures, and offers flexibility but requires careful memory management.

Likely questions

Q. Define data structure and classify it with examples.

Answer: A data structure organizes data for efficient processing. It may be primitive/non-primitive; non-primitive structures can be linear (array, stack, queue, linked list) or non-linear (tree, graph). Structures can also be static or dynamic.

Q. Explain time and space complexity.

Answer: Time complexity describes how operation count grows with input size; space complexity describes memory required. They are commonly expressed using O, Ω and Θ notation and analysed for best, average and worst cases.

Q. Explain linear-array memory representation.

Answer: Array elements occupy contiguous memory. Address(A[i]) = BA + (i – LB) × element size. This enables O(1) indexed access; insertion or deletion in the middle usually requires shifting elements.

UNIT 2 — Stacks, Recursion and Queues

Weightage 15%. Practise algorithms for push/pop, infix-to-postfix, postfix evaluation, recursion, and circular queues.

1. Stack

A **stack** is a linear data structure that follows LIFO (Last In, First Out). Insertion is called **push**; removal is **pop**; reading the top item is **peek/top**. Overflow occurs when pushing onto a full fixed-capacity stack; underflow occurs when popping an empty stack.

Array stack algorithms

```
PUSH(S, x):
  if top == capacity - 1: report OVERFLOW
  else: top = top + 1; S[top] = x

POP(S):
  if top == -1: report UNDERFLOW
  else: x = S[top]; top = top - 1; return x
```

Push, pop and peek are O(1). Initially top = –1 for a zero-based array stack.

2. Applications of stacks

• Function calls and recursion call stack.

• Undo/redo, browser history, syntax checking and backtracking.

• Expression conversion and evaluation.

3. Polish notation and expression forms

Infix: operator between operands, e.g. A + B. **Prefix (Polish):** operator before operands, e.g. +AB. **Postfix (Reverse Polish):** operator after operands, e.g. AB+.

Postfix and prefix avoid parentheses when operator order is unambiguous. Typical precedence: parentheses, exponentiation, multiplication/division/modulo, addition/subtraction. Associativity matters; exponentiation is commonly right-associative.

4. Infix to postfix conversion

Scan tokens left to right. Output operands immediately. Push '(' onto stack. For an operator, pop higher-precedence operators (and equal-precedence operators when left-associative) to output, then push the current operator. On ')', pop until '(' and discard the parentheses. At the end, pop remaining operators.

Example: Infix (A+B)*C becomes postfix AB+C*.

5. Postfix evaluation

Scan left to right. Push each operand. When an operator appears, pop the right operand first, then the left operand; apply the operator and push the result. The final stack item is the answer.

Example: 23*54*+ means (2×3)+(5×4) = 26.

6. Recursion

A function is recursive when it calls itself to solve a smaller instance of a problem. Every correct recursive solution needs a **base case** and a **recursive case**. Calls use stack memory.

```
FACTORIAL(n):
  if n <= 1: return 1
  return n * FACTORIAL(n - 1)
```

Factorial time O(n), call-stack space O(n). Naive recursive Fibonacci has exponential time O(2■) approximately; an iterative or memoized version can be O(n).

• **GCD (Euclid):** while b ≠ 0, replace (a,b) with (b, a mod b); return a. Time O(log min(a,b)) in the standard analysis.

• **Tower of Hanoi:** move n–1 disks source→auxiliary, move largest source→destination, then n–1 auxiliary→destination. Recurrence T(n)=2T(n–1)+1; moves = 2■–1; time O(2■).

7. Queue

A **queue** follows FIFO (First In, First Out). Enqueue inserts at the rear; dequeue removes from the front. A simple array queue can waste freed slots unless circular indexing is used.

8. Circular queue

A **circular queue** treats the array as a ring. With capacity N, advance an index using (index+1) mod N. In a common one-empty-slot design, empty means front = rear and full means (rear+1) mod N = front. Other implementations use a count or an explicit full flag.

9. Deque and priority queue

A **deque** (double-ended queue) allows insertion and deletion at both ends. An **input-restricted deque** restricts insertion to one end; an **output-restricted deque** restricts deletion to one end. A **priority queue** removes items according to priority rather than arrival order; heaps commonly implement it.

Queue algorithm (circular, one-empty-slot convention)

```
ENQUEUE(x):
  if (rear + 1) mod N == front: report FULL
  else: A[rear] = x; rear = (rear + 1) mod N

DEQUEUE():
  if front == rear: report EMPTY
  else: x = A[front]; front = (front + 1) mod N; return x
```

Likely questions

Q. Differentiate stack and queue.

Answer: Stack uses LIFO and insertion/removal occur at the top. Queue uses FIFO, with insertion at rear and removal at front. Both support O(1) basic operations in suitable implementations.

Q. Convert (A+B)*C to postfix.

Answer: First convert A+B to AB+; then multiply that result by C, giving AB+C*.

Q. What is recursion? Explain Tower of Hanoi complexity.

Answer: Recursion solves a problem through calls to smaller instances with a base case. Tower of Hanoi follows T(n)=2T(n–1)+1 and requires 2■–1 moves, so its time complexity is O(2■).

UNIT 3 — Linked Lists

Weightage 10%. Learn node representation and insertion/deletion at the beginning, end, and a specified position.

1. Linked list: definition

A **linked list** is a dynamic linear structure made of nodes. Each node stores data and one or more links to other nodes. Nodes need not be stored contiguously in memory.

2. Types

• **Singly linked list:** each node has data and next pointer; traversal is forward only.

• **Doubly linked list:** each node has prev and next pointers; traversal is possible in both directions but uses extra memory.

• **Circular singly linked list:** last node points to the first node.

• **Circular doubly linked list:** last and first nodes link to each other in both directions.

3. Basic node representation

```
Node:
  data
  next
```

The head pointer refers to the first node. An empty list has head = NULL. A tail pointer can make insertion at the end O(1), if maintained correctly.

4. Singly linked-list operations

Traversal: start at head and follow next until NULL; O(n). **Search:** compare each node; O(n). **Insert at head:** new.next=head; head=new; O(1). **Delete head:** head=head.next; O(1), after checking non-empty list.

```
INSERT_HEAD(head, x):
  n = new Node(x)
  n.next = head
  head = n
  return head

SEARCH(head, key):
  p = head
  while p != NULL:
    if p.data == key: return p
    p = p.next
  return NOT_FOUND
```

Insertion after a known node is O(1). Insertion at the end is O(n) without a tail pointer, O(1) with a maintained tail pointer. Deleting a node generally requires locating its predecessor in a singly linked list.

5. Doubly linked-list operations

For insertion between nodes left and right, update all four links: new.prev=left; new.next=right; left.next=new; right.prev=new, while handling head/tail boundaries. Deletion reconnects the predecessor and successor. With a pointer to the target node, deletion is O(1) if links and boundaries are maintained.

6. Memory allocation and garbage collection

Linked-list nodes are commonly allocated dynamically. **Memory allocation** reserves memory for a node. **Garbage collection** reclaims memory that is no longer reachable or used; in languages without automatic garbage collection, the programmer must release nodes appropriately.

7. Linked list vs array

Arrays provide O(1) indexed access and contiguous storage; linked lists require O(n) traversal for the i-th element. Linked lists can insert/delete at a known position in O(1), while arrays may shift O(n) elements. Linked lists require pointer overhead and have poorer cache locality.

8. Implementing stack and queue using linked lists

For a stack, push/pop at the head give O(1) operations. For a queue, maintain front and rear pointers: enqueue at rear and dequeue at front in O(1). Always update both pointers when the queue becomes empty.

Likely questions

Q. Explain singly, doubly and circular linked lists.

Answer: Singly nodes store next; doubly nodes store previous and next; circular lists connect the final node back to the first. Doubly lists support reverse traversal but require extra memory.

Q. Give advantages and disadvantages of linked lists.

Answer: Advantages: dynamic size and efficient insertion/deletion when the location is known. Disadvantages: no direct indexing, pointer memory overhead, more complex implementation and weaker cache locality.

Q. How can a queue be implemented using a linked list?

Answer: Maintain front and rear. Enqueue by linking a new node after rear and updating rear. Dequeue by removing front and moving front to front.next. Both operations are O(1); if front becomes NULL, set rear to NULL.

UNIT 4 — Searching and Sorting

Weightage 10%. Memorize the preconditions and complexity of each search/sort.

1. Searching

Linear search: inspect elements one by one until the key is found or the list ends. Works on unsorted data. Best O(1), average/worst O(n).

```
LINEAR_SEARCH(A, key):
  for i = 0 to n-1:
    if A[i] == key: return i
  return -1
```

Binary search: repeatedly halves the search interval. The array must be sorted. Time O(log n), iterative extra space O(1).

```
BINARY_SEARCH(A, key):
  low = 0; high = n - 1
  while low <= high:
    mid = low + (high - low) // 2
    if A[mid] == key: return mid
    if A[mid] < key: low = mid + 1
    else: high = mid - 1
  return -1
```

Interpolation search: estimates a likely position based on the key's value. It works best on sorted, approximately uniformly distributed numeric data. Average O(log log n) under favourable distribution assumptions; worst O(n).

2. Sorting algorithms

Selection sort: repeatedly select the minimum from the unsorted suffix and swap it into place. Time $O(n^2)$ in all cases; in-place; usually unstable.

Insertion sort: take each next element and insert it into the correct place in the sorted prefix. Best O(n) on nearly/already sorted input; average/worst $O(n^2)$; stable and in-place.

Bubble sort: repeatedly swap adjacent out-of-order elements. With an early-stop flag, best O(n), average/worst $O(n^2)$; stable and in-place.

Merge sort: divide array into halves, sort recursively, merge sorted halves. Time O(n log n); typical array implementation needs O(n) auxiliary memory; stable.

Quick sort: partition around a pivot and recursively sort partitions. Average O(n log n), worst $O(n^2)$ with poor partitions; usually in-place apart from recursion; generally unstable.

3. Sorting comparison table

Algorithm	Best	Average	Worst	Stable?	Extra space (typical)
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	O(1)
Insertion	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)
Bubble (optimized)	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	Yes	O(n)
Quick	O(n log n)	O(n log n)	$O(n^2)$	Usually no	O(log n) average recursion

Likely questions

Q. Why must binary search use sorted data?

Answer: It discards half the search interval based on ordering. Without sorted order, comparing the middle element does not tell which half can contain the key.

Q. Compare merge sort and quick sort.

Answer: Both average O(n log n). Merge sort guarantees O(n log n), is stable, and typically needs O(n) extra array memory. Quick sort is often fast in practice and typically in-place, but can degrade to $O(n^2)$ with poor pivots.

Q. Explain interpolation search.

Answer: It estimates the key's position using the values at low and high indices, rather than always choosing the midpoint. It is most effective for uniformly distributed sorted numeric data and can have O(n) worst-case time.

UNIT 5 — Trees and Binary Search Trees

Weightage 15%. Focus on terminology, representations, traversals, BST operations, height, and expression trees.

1. Tree definitions and terminology

A **tree** is a hierarchical, connected structure with no cycles. The **root** is the top node. A **parent** has child nodes; **sibling** nodes share a parent. A **leaf** has no children. An **internal node** has at least one child. An **edge** connects nodes. A node's **depth** is its distance in edges from the root; a node's **height** is the longest downward path to a leaf. Tree height is the root's height.

A **binary tree** has at most two children per node, called left and right. A **full/strict** binary tree has either zero or two children per node. A **complete** binary tree fills levels from left to right, with only the last level possibly incomplete. A **perfect** binary tree has all internal nodes with two children and all leaves at the same level.

2. Binary-tree properties

• Maximum nodes at level ■, with root at level 0: 2^■.

• Maximum nodes in a binary tree of height h (height measured in edges): 2^(h+1) – 1.

• A perfect binary tree of height h has 2^(h+1) – 1 nodes and 2^h leaves.

• For any non-empty tree, number of edges = number of nodes – 1.

3. Binary-tree representation

Array representation: for zero-based indexing, left child of i is 2i+1, right child is 2i+2, and parent is floor((i–1)/2). Efficient for complete trees but can waste space for sparse trees.

Linked representation: each node stores data, left pointer and right pointer. Flexible for sparse or changing trees.

4. Tree traversals

Preorder: Root, Left, Right. **Inorder:** Left, Root, Right. **Postorder:** Left, Right, Root. **Level order:** visit nodes level by level using a queue.

```
PREORDER(node):
  if node != NULL:
    visit(node)
    PREORDER(node.left)
    PREORDER(node.right)

INORDER(node):
  if node != NULL:
    INORDER(node.left)
    visit(node)
    INORDER(node.right)

POSTORDER(node):
  if node != NULL:
    POSTORDER(node.left)
    POSTORDER(node.right)
    visit(node)
```

Each traversal visits every node once: O(n) time. Recursive auxiliary stack is O(h), where h is tree height.

5. Binary Search Tree (BST)

A **BST** is a binary tree where keys in the left subtree are smaller than the node's key and keys in the right subtree are larger, under the chosen duplicate-key policy. Search, insertion and deletion take O(h), where h is tree height; balanced trees have O(log n), while a skewed tree can have O(n).

BST search: compare the key with the current node; go left if smaller and right if larger. **Insertion:** follow the same comparisons until a null child is found. **Deletion:** (1) leaf: remove it; (2) one child: replace node with child; (3) two children: replace key with inorder successor (minimum of right subtree) or predecessor, then delete that replacement node.

6. Expression tree

An **expression tree** stores operators in internal nodes and operands in leaves. Preorder produces prefix form, inorder produces infix form (parentheses may be needed), and postorder produces postfix form.

7. Balanced binary tree

A **balanced tree** keeps height proportional to log n so search and update operations remain efficient. Balance conditions vary by tree type; AVL and Red-Black trees are self-balancing BSTs.

Likely questions

Q. Write the three depth-first traversals.

Answer: Preorder: Root–Left–Right; Inorder: Left–Root–Right; Postorder: Left–Right–Root. Each visits every node once, so time is O(n).

Q. Explain deletion in a BST.

Answer: If the node is a leaf, remove it. If it has one child, link its parent directly to the child. If it has two children, copy the inorder successor or predecessor's key and delete that successor/predecessor from its original position.

Q. Differentiate full, complete and perfect binary trees.

Answer: Full: every node has zero or two children. Complete: all levels except possibly the last are full and the last is filled from left to right. Perfect: every internal node has two children and all leaves are at the same level.

UNIT 6 — AVL Trees and Red-Black Trees

Weightage 10%. Practise AVL balance factors and rotations; understand Red-Black invariants and why balancing matters.

1. AVL tree

An **AVL tree** is a self-balancing binary search tree. For every node, the balance factor is height(left subtree) – height(right subtree), and it must be –1, 0, or +1.

2. AVL rotations

• **LL case:** imbalance caused by insertion into left subtree of left child; perform a single right rotation.

• **RR case:** insertion into right subtree of right child; perform a single left rotation.

• **LR case:** insertion into right subtree of left child; left-rotate the left child, then right-rotate the unbalanced node.

• **RL case:** insertion into left subtree of right child; right-rotate the right child, then left-rotate the unbalanced node.

AVL search, insertion and deletion are O(log n) because the height remains logarithmic. Insertions/deletions may require updating heights and rotations.

3. AVL insertion procedure

Insert the key as in a normal BST. Update ancestor heights while returning toward the root. Compute each balance factor; if it becomes +2 or –2, identify LL, RR, LR or RL from the inserted key's path and rotate accordingly.

4. Red-Black tree

A **Red-Black tree** is a BST with node colours and rules that keep the tree approximately balanced. The standard properties are:

• Every node is red or black.

• The root is black.

• All NIL/null leaves are considered black.

• A red node cannot have a red child (no two consecutive red nodes).

• Every path from a node to a descendant NIL leaf contains the same number of black nodes.

These properties guarantee height O(log n), so search, insertion and deletion are O(log n). Insertion/deletion use recolouring and rotations to restore invariants.

5. AVL vs Red-Black

AVL trees enforce stricter height balance, often giving faster lookups but potentially more rebalancing during updates. Red-Black trees permit looser balance, often reducing update rebalancing. Both support O(log n) search, insertion and deletion.

Likely questions

Q. What is the balance factor of an AVL node?

Answer: Balance factor = height(left subtree) – height(right subtree). In a valid AVL tree it must be –1, 0 or +1 for every node.

Q. Explain LL, RR, LR and RL rotations.

Answer: LL is fixed by a right rotation; RR by a left rotation; LR by left rotation on the left child followed by right rotation; RL by right rotation on the right child followed by left rotation.

Q. State Red-Black tree properties.

Answer: Nodes are red or black; root and NIL leaves are black; red nodes have black children; every node-to-descendant-NIL path has the same black height. These invariants guarantee O(log n) height.

UNIT 7 — Hashing

Weightage 15%. High priority: hash functions, collision handling, static/dynamic hashing, and table organisation.

1. Hashing and hash table

Hashing maps a key to an index using a hash function. A **hash table** stores records in slots or buckets addressed by that index. Average search/insert/delete can be O(1) with a good hash function and controlled load factor; worst case can be O(n).

2. Hash function

A **hash function** maps keys to table indices 0 through m–1. A good function is fast and distributes keys uniformly. A simple division method is h(k)=k mod m; choosing a suitable table size helps distribution.

3. Collision

A **collision** occurs when two different keys map to the same index. Collision resolution methods include chaining and open addressing.

4. Separate chaining

Each table slot points to a collection (often a linked list) of keys hashing to that slot. Insert/search examines the corresponding chain. Expected performance is good when chains remain short; worst-case search is O(n).

5. Open addressing

All keys are stored in the table itself. When a collision occurs, probe alternative slots.

• **Linear probing:** h(k,i)=(h(k)+i) mod m. Simple but causes primary clustering.

• **Quadratic probing:** h(k,i)=(h(k)+c1i+c2i²) mod m. Reduces primary clustering but requires suitable parameters/table size.

• **Double hashing:** h(k,i)=(h1(k)+i·h2(k)) mod m. A second hash chooses the step size; h2(k) must be nonzero and suitably coprime to m.

Open-addressing deletion commonly uses a tombstone/deleted marker rather than making a slot simply empty, because an empty slot could break a later search's probe sequence.

6. Load factor

Load factor α = number of stored keys / table capacity. For chaining, α can exceed 1; for open addressing, α must remain below 1. As α rises, collisions and probe lengths generally increase. Resizing and rehashing can restore performance.

7. Static and dynamic hashing

Static hashing: table/bucket structure is fixed. It is simple but may perform poorly as data volume changes. **Dynamic hashing:** expands or reorganizes buckets as records grow; examples include extendible hashing and linear hashing.

8. Example: division hash

For m=10 and key k=42, h(k)=42 mod 10=2. If key 32 also maps to 2, a collision occurs; chaining stores both in bucket 2, while linear probing checks the next available slot.

Likely questions

Q. Define hashing and explain collision resolution.

Answer: Hashing computes a table index from a key. A collision occurs when keys share an index. Common solutions are separate chaining, linear probing, quadratic probing and double hashing.

Q. Compare chaining and open addressing.

Answer: Chaining stores collided keys in a collection at each bucket and can have load factor above 1. Open addressing stores all keys in the table and probes for free slots; it requires capacity and careful deletion handling.

Q. What is load factor?

Answer: $\alpha = n/m$, where n is the number of stored keys and m is table capacity. A high load factor generally increases collision cost; resizing/rehashing can improve performance.

UNIT 8 — Graphs

Weightage 15%. Learn graph terminology, representations, basic operations, BFS and DFS algorithms.

1. Graph: definition

A **graph** G=(V,E) consists of a set of vertices V and edges E connecting pairs of vertices. In a directed graph, edges have direction; in an undirected graph, edges do not. A weighted graph assigns a weight or cost to edges.

2. Graph terminology

• **Adjacent vertices:** vertices connected by an edge.

• **Degree:** number of incident edges in an undirected graph. In a directed graph, indegree counts incoming edges and outdegree counts outgoing edges.

• **Path:** sequence of vertices connected by edges. **Cycle:** a path that returns to its starting vertex.

• **Connected graph:** an undirected graph with a path between every pair of vertices. A directed graph is strongly connected if every vertex can reach every other vertex.

• **Subgraph:** graph formed from subsets of vertices and edges. **Complete graph:** every pair of distinct vertices has an edge.

3. Graph representation

Adjacency matrix: V×V matrix; entry indicates whether an edge exists (or stores its weight). Space $O(V^2)$; checking an edge is O(1); iterating neighbours of a vertex takes O(V). Good for dense graphs.

Adjacency list: each vertex stores its neighbours. Space O(V+E); iterating neighbours is efficient. Good for sparse graphs.

4. Breadth-First Search (BFS)

BFS explores a graph level by level from a starting vertex using a queue. Mark a vertex visited when it is enqueued to avoid repeated insertion. BFS finds shortest paths by number of edges in an unweighted graph.

```
BFS(G, start):
  mark start visited; enqueue(start)
  while queue not empty:
    u = dequeue()
    visit(u)
    for each neighbour v of u:
      if v not visited:
        mark v visited
        parent[v] = u
        enqueue(v)
```

With an adjacency list, BFS takes O(V+E) time and O(V) auxiliary space.

5. Depth-First Search (DFS)

DFS explores as far as possible along a branch before backtracking. It can be implemented recursively or with an explicit stack.

```
DFS(G, u):
  mark u visited
  visit(u)
  for each neighbour v of u:
    if v not visited:
      parent[v] = u
      DFS(G, v)
```

With an adjacency list, DFS takes O(V+E) time and O(V) auxiliary space including visited/recursion storage. To traverse a disconnected graph fully, start a traversal from every unvisited vertex.

6. BFS vs DFS

Feature	BFS	DFS
Main structure	Queue	Stack or recursion
Exploration	Level by level	Deep branch then backtrack
Unweighted shortest path	Yes	Not guaranteed
Typical uses	Shortest path, levels, connectivity	Cycle detection, components, topological ordering in DAGs
Adjacency-list complexity	O(V+E)	O(V+E)

7. Elementary graph operations

Create vertices/edges, remove vertices/edges, check adjacency, find neighbours, traverse vertices, and search for a path. The cost depends on representation: for example, checking an edge is O(1) in an adjacency matrix but typically O(degree(u)) in an unsorted adjacency list.

Likely questions

Q. Compare adjacency matrix and adjacency list.

Answer: Matrix uses $O(V^2)$ space and checks an edge in O(1), making it suitable for dense graphs. List uses O(V+E) space and efficiently iterates neighbours, making it suitable for sparse graphs.

Q. Explain BFS with complexity.

Answer: BFS uses a queue and visits vertices in increasing distance from the start. Mark each vertex when enqueued, then enqueue unvisited neighbours. With an adjacency list it runs in O(V+E).

Q. Explain DFS with complexity.

Answer: DFS visits a vertex and recursively explores each unvisited neighbour before backtracking. With an adjacency list it runs in O(V+E). It is useful for reachability, connected components and cycle-related tasks.

Final Revision — Common Comparisons and Exam Checklist

Must-memorize complexity table

Operation/algorithm	Typical time
Array indexed access	O(1)
Array search (unsorted)	O(n)
Stack push/pop/peek	O(1)
Queue enqueue/dequeue	O(1) with suitable implementation
Linked-list traversal/search	O(n)
Binary search	O(log n)
Linear search	O(n)
Selection / insertion / bubble sort	$O(n^2)$ average/worst
Merge sort	O(n log n)
Quick sort	O(n log n) average; $O(n^2)$ worst
BST search/insert/delete	O(h); O(log n) balanced, O(n) skewed
AVL / Red-Black operations	O(log n)
Hash table operations	O(1) average; O(n) worst
BFS / DFS with adjacency list	O(V+E)

Last-day checklist

• I can define every bold term in 2–3 sentences.

• I can write push/pop, circular queue, linked-list insertion, binary search, traversals, BFS and DFS pseudocode.

• I can convert infix to postfix and evaluate a postfix expression by hand.

• I can explain BST deletion's three cases and all four AVL rotations.

• I can solve a hashing collision example using chaining and linear probing.

• I can state the complexity and precondition of every search/sort algorithm.

• I can compare stack vs queue, array vs linked list, BFS vs DFS, matrix vs list, AVL vs Red-Black.

Exam writing tip: For a 5–7 mark answer, use this order: definition → diagram/example → algorithm or properties → time/space complexity → advantages/applications. Draw a neat labelled diagram wherever possible.

These notes cover the topics shown in the uploaded syllabus. Your lecturer's preferred notation and implementation conventions may differ, so follow class notes where they specify a particular convention.